Elementary Data Structures

Chapter 1

Instructor: Shailesh B. Pandey
Everest Engineering College

2024

Natural Language

- Language in use has a sequential order
  - Words are uttered/written in a sequence: I bough a book on data structures

- It also exhibits a hierarchical order (phrases) and dependencies
  - सुरेशले मिठो गीत गाए
  - सुनिताले मिठो गीत गाइन्

- Data structures such as strings, lists, trees, and graphs are commonly used

Data Structures for NLP

- Unprocessed data is usually input as a string data
  - Processed into lists or vectors representing individual words

- Subsequent processing in the NLP pipeline produces complex objects
  - Need to track attributes associated with each word (or token)
    - e.g. parts of speech (Noun, Adjective etc.), sentiment (positive/negative/neutral) etc.
  - Later stages add additional annotations
    - markings to denote beginning and endings of important sequences in a sentence
    - e.g. [Everest Engineering College] is located in [Sanepa]

Strings

Operations:
Tokenization, substring search, replacement, concatenation.

- They are sequence of characters
- Primary way to represent unprocessed data
  - A string can store an entire document, a sentence, or a single word
- Important operations for strings include: concatenation (combining two strings into one), matching substrings (e.g. to check for suffix or prefix), finding length of a string (e.g. very short string might be an abbreviation)
  - Tokenization, substring search, replacement
- Regular expression is the workhorse for string operations

Lists

Operations: Sorting, slicing, filtering (e.g., stopword removal).

- Lists and vectors are both ordered collection of elements
- Lists are used to hold sequences where the sizes of element might vary, such as sequence of words
- List can also have more or less elements over time
- Lists that are immutable during the execution of a program are sometimes called tuples

Lists

- Important operations over the list include finding individual pattern or patterns within the list

    - [{'LIKE_NUM': True, 'OP': '?’},
      {'POS': 'ADJ', 'OP': '?’},
      {'LEMMA': 'duck', 'POS': 'NOUN’}]

- The pattern in spaCy matches phrase such as : three yellow ducks, one happy duck etc.

- [{"ORTH": "("}, {"SHAPE": "ddd"}, {"ORTH": ")"},
  {"SHAPE": "ddd"},{"ORTH": "-", "OP": "?"},
  {"SHAPE": "dddd"}]

Matches (977) 123-4567

Maps and Sets

- Dictionaries (Hash Maps)
  - Purpose: Map words to associated values, such as counts, probabilities, or embeddings.
  - Example Data : word = {"the": 10, "quick": 3, "fox": 5} word_to_index = {"the": 0, "quick": 1, “fox": 2}
  - Operations: Word frequency count, word-to-index mapping for embedding lookup.

- Sets
  - Purpose: Store unique words or characters to eliminate duplicates.
  - Example Data: {"quick", "fox", "jumps"}
  - Operations: Removing duplicates, finding intersections (common words) between texts.

Vectors

- Vectors hold elements of the same size, like numbers, and are of fixed size
  - They are one-dimensional
- The generalization of matrices to N-dimensional space is called a tensor
- Important operations include: accessing an element, finding length, similarity, taking averages

Vectors and NLP

- Vectors can represent subsets of words or labels from a fixed vocabulary
  - Value is either zero or one
- Vectors where only one element is zero or one is used as an output format
  - e.g. to indicate whether the word is a verb or a noun
- Vectors can also capture a probability distribution over a set
  - e.g. classification

One Hot Encoding

- Often we treat words as discrete symbols

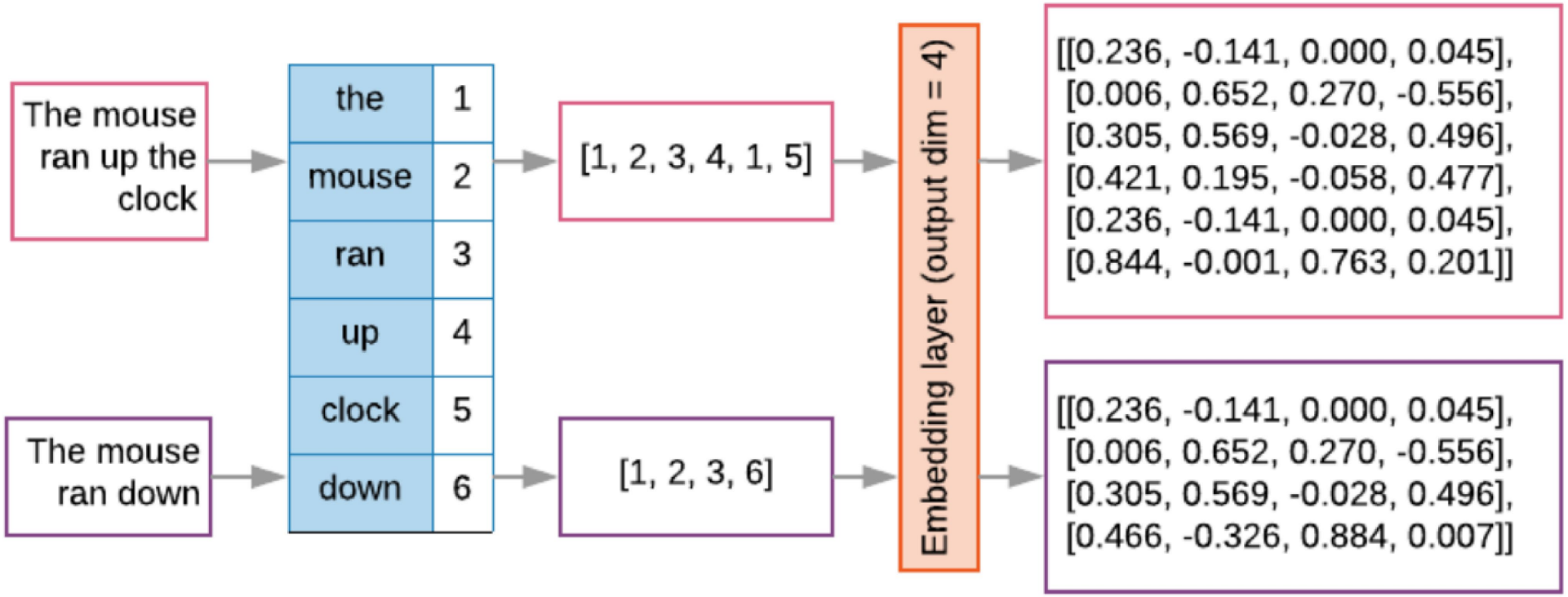
- Words can be represented by one-hot vectors:

  hotel = [0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]

  motel = [0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0]

Matrix

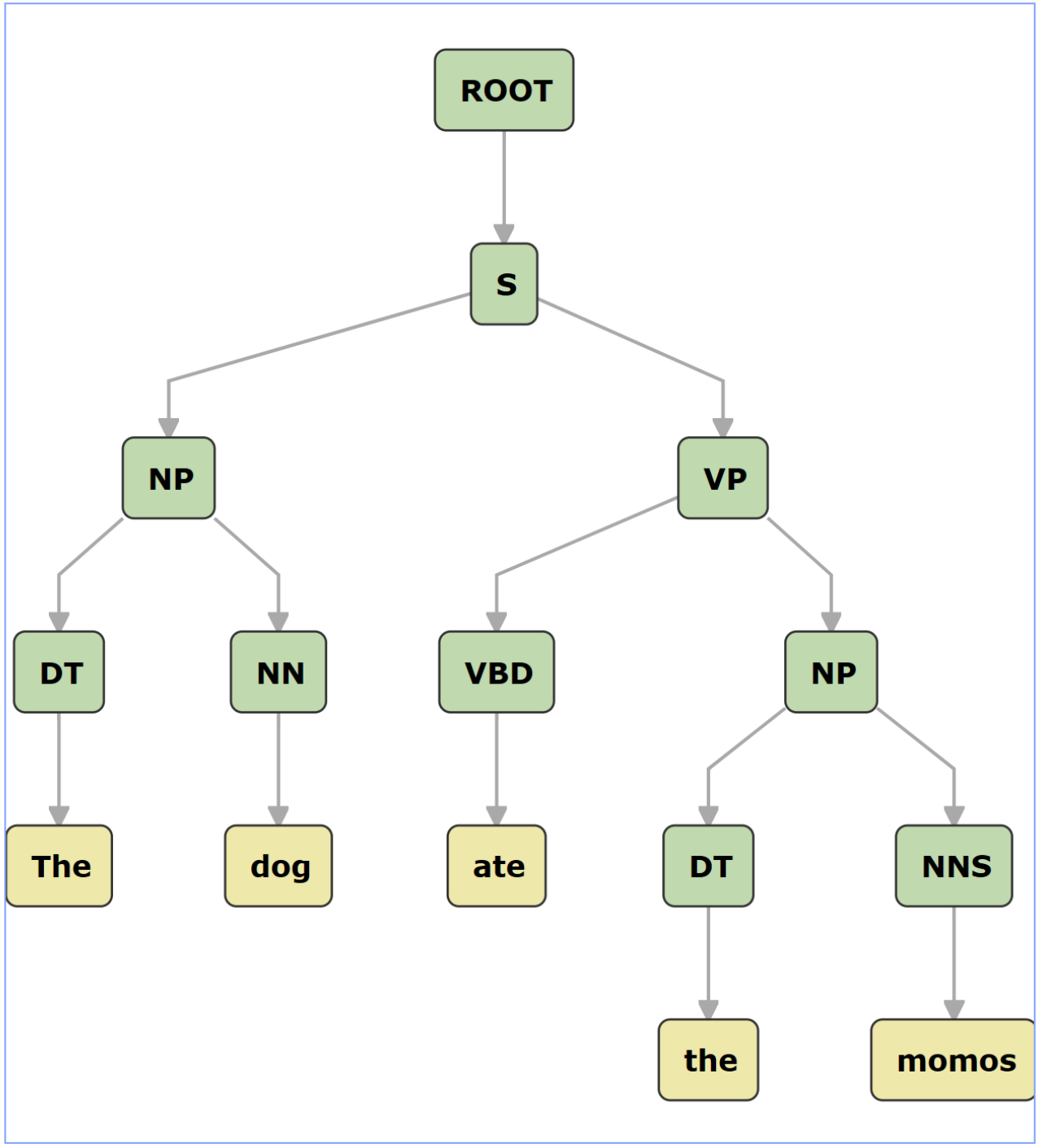
- Sequence models often have an embedding layer as their first layer. This layer learns to turn word index sequences into word embedding vectors during the training process, such that each word index gets mapped to a dense vector of real values representing that word's location in semantic space



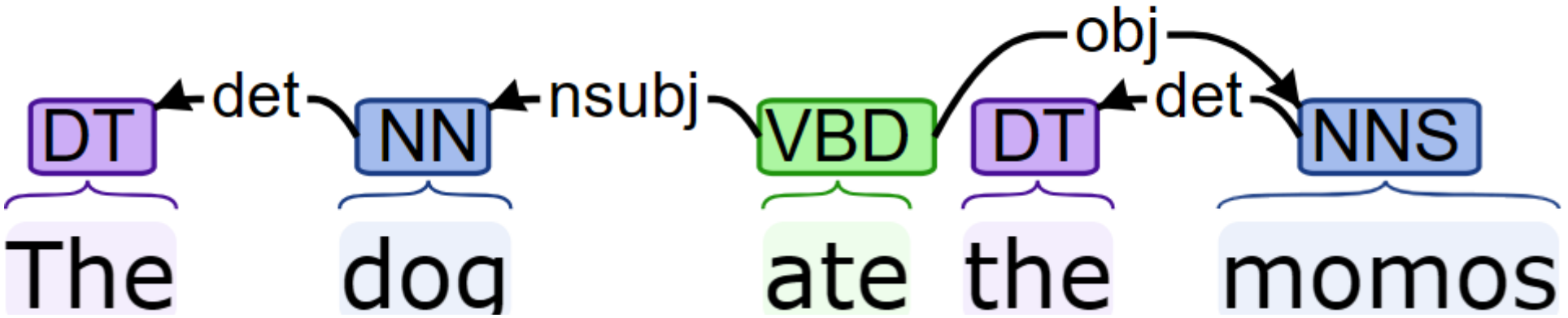
Trees

- Trees are specified by set of nodes and connection between pairs of nodes, called edges
- In a tree edges are directed and each node has a unique predecessor (the parent) and multiple successors (the children)
- Trees are used to represent the syntactic structure of sentences

Constituency



Dependency



Correspondence: List and Trees

- There is a direct correspondence between nested lists and trees, so both can be used to represent syntactic structures of sentence.

- The structure for the sentence shown in the constituency parse tree is the same as the following nested list:

- (S (NP (DT the) (NN dog))
  (VP (VBD ate) (NP (DT the) (NNS momos))))

- [(DT, the, 0, 1), (NN, dog, 1, 2), (VBD, ate, 2,3),(DT, the, 3, 4),
  (NNS, momos, 4, 5), (NP, 0, 2), (NP, 3,5), (VP, 2, 5), (S, 0, 5)]

Graphs

- Graphs are more general than trees
  - nodes can have multiple incoming edges
- In search, the node corresponds to a state
- In a Neural Network, the node corresponds to a operation or function, with any number of inputs and outputs
- Edges can be scalars, vectors, matrices …
- In both search graphs and neural network, the nodes and edges represent models of process rather than data

Knowledge Representation

- Suresh knows Rabin
- Suresh knows Muna
- Rabin knows Nabina

- Suresh is 35 years old
- Rabin is 27 years old
- Muna is 25 years old
- Nabina is 29 years old

Knowledge Representation

knows

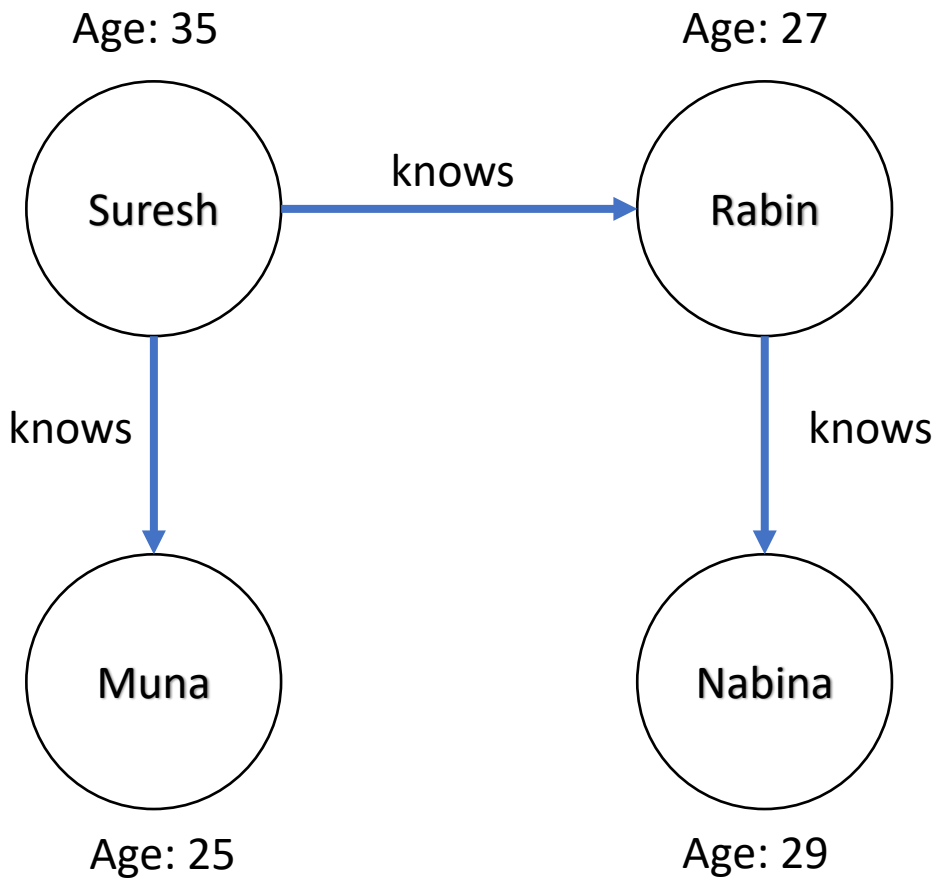
Suresh	Rabin
Suresh	Muna
Rabin	Nabina

age

Suresh	35
Rabin	27
Muna	25
Nabina	29

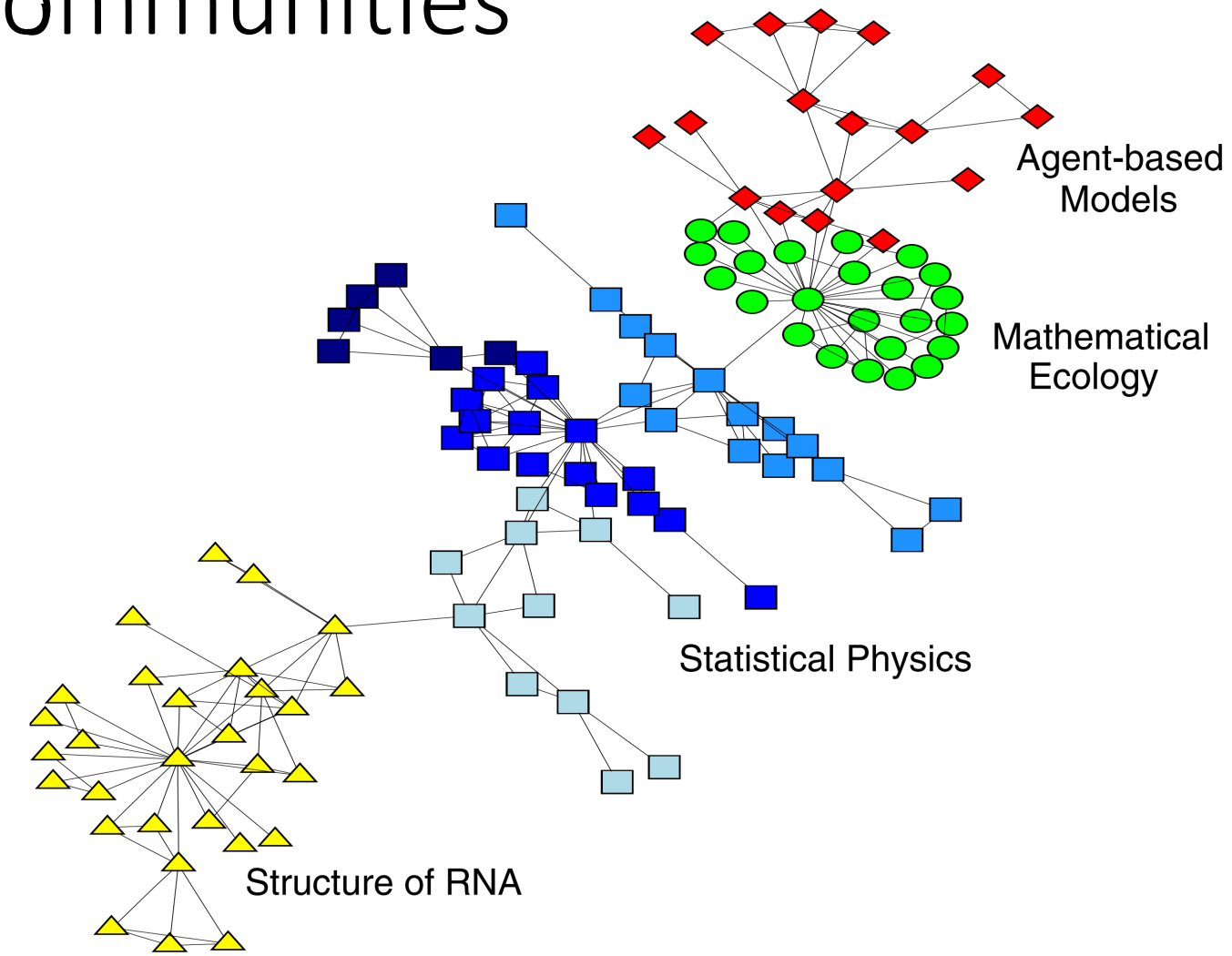
Relational Database Representation

Knowledge Representation

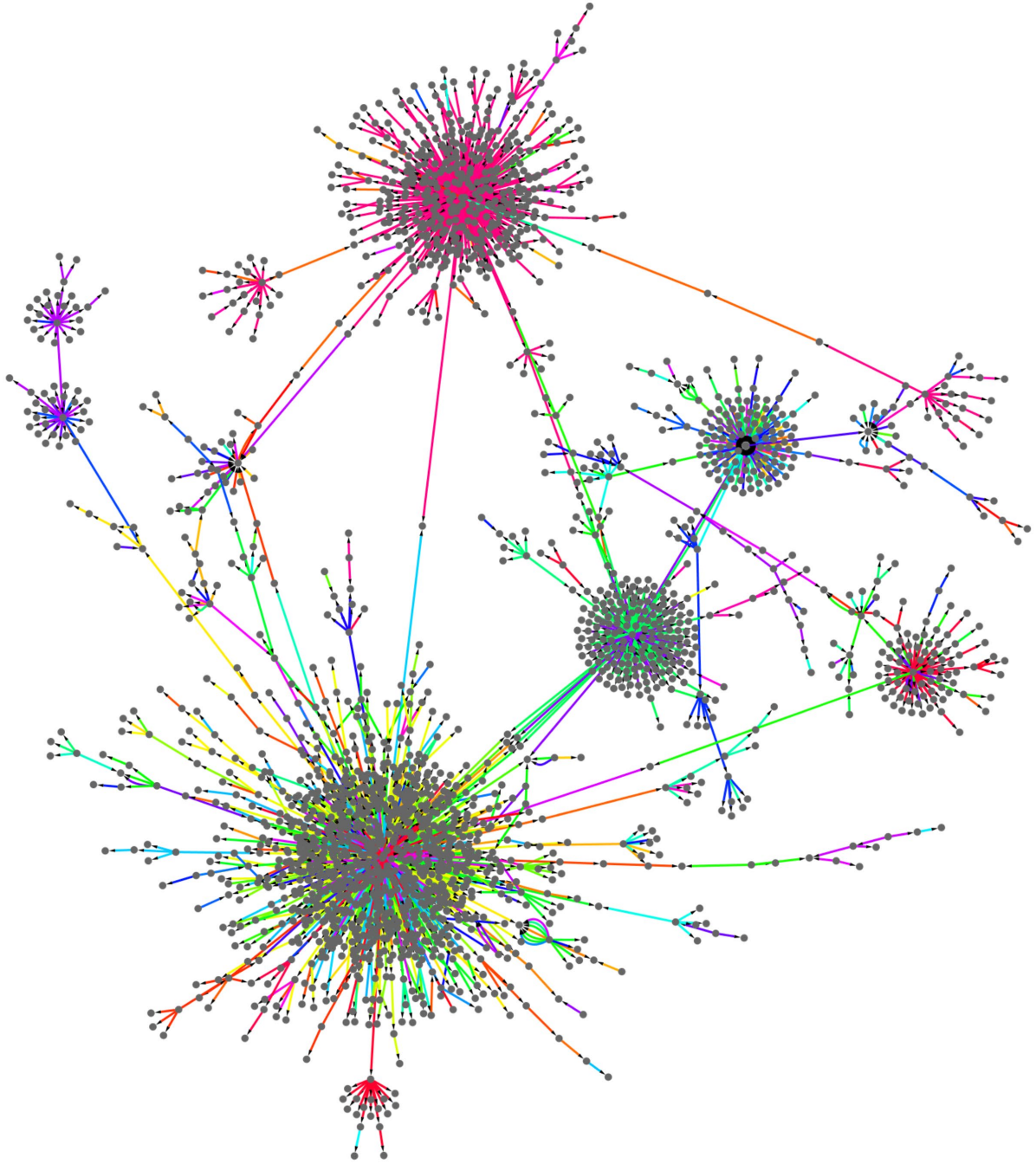


Attributed Graph Representation

Networks and Communities



Retweet Trees



Case Study

- Vector Space Model

- Retweets

Data Structures

- Primary purpose of most computer programs is to store and retrieve information, not to perform calculations

- A good data structure simplifies the algorithm

Vending Machine

```
public class VendingMachineInventory{

public static void main(String[] args){
int cocacola = 0;
int pepsi = 0;
int sprite = 0;

 java.util.Scanner console = new java.util.Scanner(System.in);
  System.out.println("Enter soft-drinks and quantity:");

 while (console.hasNext()){
    String soda = console.next().toUpperCase();
    int quantity = console.nextInt();
     char response = soda.charAt(0);

     if (response == 'C') cocacola += quantity;
     if (response == 'P') pepsi += quantity;
     if (response == 'S') sprite += quantity;
   }

    System.out.println("Final inventory.\n");
    System.out.println("coca-cola " + cocacola);
    System.out.println("pepsi " + pepsi);
    System.out.println("sprite " + sprite);
   }
}
```

Issues

- The program works correctly but the it doesn't scale up very easily
- If we want to add a fifth variety, like “Diet Coke", we have to make modifications to three different parts of the code:
  - add a new variable
  - add another if statement
  - add another print statement to the report
- Each modification introduces the possibility of making a mistake

Issues

- In addition, certain kinds of computations become quite difficult to express:
- For example, how would we find the average inventory?
  (coke + dietcoke + pepsi + sprite) / 4

- When a fifth soft drink is added, one must remember to change the denominator as well as add the new item in the numerator

- What if we want to add five more soft drinks, or ten? At some point, the code just gets too unwieldy to manage
  - It's just not practical to store every data item in its own separately named variable

Vending Machine with Arrays

```
public class VendingMachineWithArrays
{
  public static void main(String[] args)
  {
   int[] inventory = new int[256];
   String sdcodes = " CPSM";

   java.util.Scanner console = new
                                java.util.Scanner(System.in);
   System.out.println("Enter soft-drinks and amount:\n");

   while (console.hasNext()){
    String softdrinks = console.next().toUpperCase();
    int amount = console.nextInt();
    char response = softdrinks.charAt(0);
    inventory[(int)response] = inventory[(int)response]
    + amount;
   }

   System.out.println("Final inventory.\n");
   for (int i = 1; i < sdcodes.length(); i++){
    char softdrinks = sdcodes.charAt(i);
    System.out.print(softdrinks +" ");
    System.out.println(inventory[(int)softdrinks]);
   }
  }
}
```

Chess Board – 1D Array

```
char[] board = {'R','N','B','Q','K','B','N','R','P','P','P','P','P','P','P','P',
                ' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',
                ' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',
                ' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',
                ' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',
                ' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',
               'p','p','p','p','p','p','p','p','r','n','b','q','k','b','n','r'};

 public boolean isEmpty(int row, int column)
  {
     int index = row * 8 + column;
     char square = board.charAt(index);
     return square == ' ';
  }
}
```

Issues

- The string is not easy to visually verify
- The algorithm to determine if a square is empty is long and difficult to understand

Contiguous vs. Linked Data Structures

- Data structures can be neatly classified as either contiguous or linked depending upon whether they are based on arrays or pointers:

- Contiguously-allocated structures are composed of single slabs of memory, and include arrays, matrices, heaps, and hash tables

- Linked data structures are composed of multiple distinct chunks of memory bound together by pointers, and include lists, trees, and graph adjacency lists.

Arrays

- An array is a structure of fixed-size data records such that each element can be efficiently located by its index or (equivalently) address.

- Advantages of contiguously-allocated arrays include:
  - Constant-time access given the index.
  - Arrays consist purely of data, so no space is wasted with links or other formatting information.
  - Physical continuity (memory locality) between successive data accesses helps exploit the high-speed cache memory on modern computer architectures.

Arrays

- If the first element of an array has index *s*
  - the array starts at memory address *a*
  - each element occupies *b* bytes
  - then the ith element occupies bytes : **a + b(i-s)** through **a + b(i-s+1)-1**

- Let us take the case of 1-origin, i.e., s=1, a=1000, b=4
  - The location of the third element is:
  - a+b(1-s) = 1008
  - a+b(i-s+1)-1 = 1011

Why Arrays?

- **Constant-time access**: index maps directly to a particular memory address

- **Space efficiency**: The array contains solely data, no space is wasted with other information. Not even end-of-record information as they are fixed-size

- **Memory locality**: Many programming tasks require iterating through the elements and physical continuity between successive data accesses helps exploit the high-speed cache memory

Limitation?

- Cannot adjust array's size in the middle of program execution.
  - Can compensate by allocating extremely large arrays, but this will waste space
- We can enlarge an array as we need it: dynamic arrays
  - We start with size 1 and double its size from m to 2m whenever we run out of space: create a new array of size 2m, copy the old array, free the old memory
- The waste is the copying the old content

Array: Time Complexity

- What is the time complexity of an insert operation?
- What is the time complexity of a delete operation?
- What is the time complexity of sequential search?
- What is the time complexity of binary search?

Dynamic Expansion

- Suppose we start with a size of 1
- After we insert an element, the size doubles to 2
- After we insert the 2nd element, the size doubles to 4
- After we insert the 4th element, the size doubles to 8
- After j^{th} doubling the size of the array is 2^{j-1}

Dynamic Expansion

- Now consider this questions: How many times do we have to double to accommodate ‘n’ elements?

- Let us assume the final size is 8

5							
5	7						
5	7	11	29				
5	7	11	29				

Dynamic Expansion

- The number of doublings is $\log_2 n$
- How?
- The array doubles in size until it reaches a capacity of 2^j, where $2^j \geq n$
- Let us check this in Excel

Dynamic Expansion

- Next, consider this question: How often do we copy an element until we reach an array of size n?

Hence

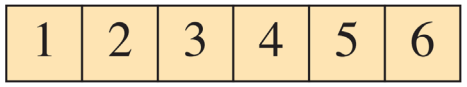
- The number of copy operations at the ith doubling will be 2^{i-1}, so the total number of movements M will be:

$$M = n + \sum_{i=1}^{\ln n} 2^{i-1}$$

Matrices

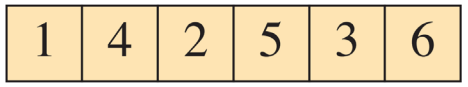
- We typically represent a matrix or two-dimensional array by one or more one-dimensional arrays

- Four ways to store a 2x3 matrix:



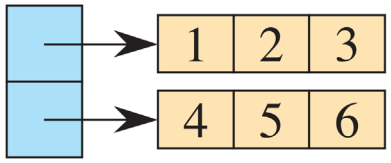
(a)

row-major order



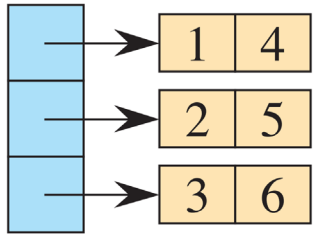
(b)

column-major order



(c)

one array per-row
one array for pointers

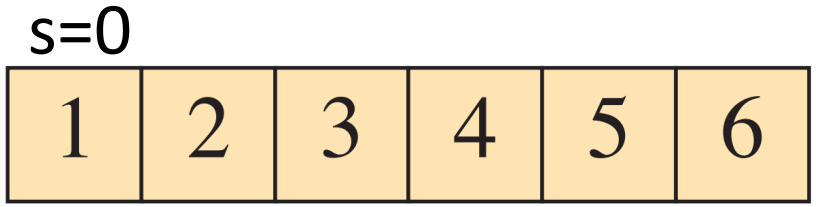


(d)

one array per-column
one array for pointers

Question

- What is the index of the second-row first-column element (here element with value 4) if the index starts at zero and we are using the row-major order for a m x n matrix?



Matrices

- If the rows, columns, and the single array all are indexed starting at s then M[i,j] - element in row i and column j - is at array index:

- Row-order
  - s + (n(i-s)) + (j-s)

- Column-order
  - s + (m(j-s)) + (i-s)

Question

- In many real-world applications the matrix is often sparse. This means that most of the elements are zero (no value) and a few have values. How can we save space? What can be the representations?

$$\begin{bmatrix} 6 & 0 & 0 & 2 & 0 & 5 \\ 4 & 4 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

Answer

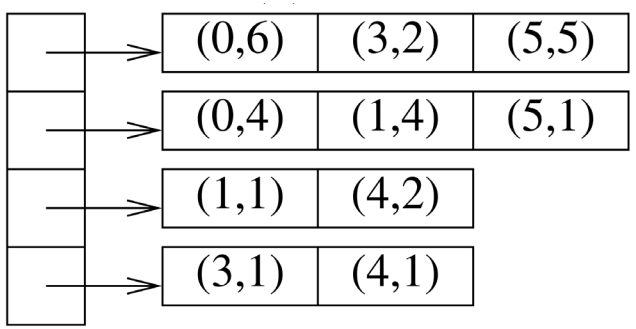
```
typedef struct {
    int row;     // Row index
    int col;     // Column index
    int value;   // Nonzero value
} Triple;
```

- Array of triples

row	0	0	0	1	1	1	2	2	3	3
col	0	3	5	0	1	5	1	4	3	4
val	6	2	5	4	4	1	1	2	1	1

```
Triple *triples = (Triple *)malloc(nonzeroCount * sizeof(Triple));
```

- Array of pairs

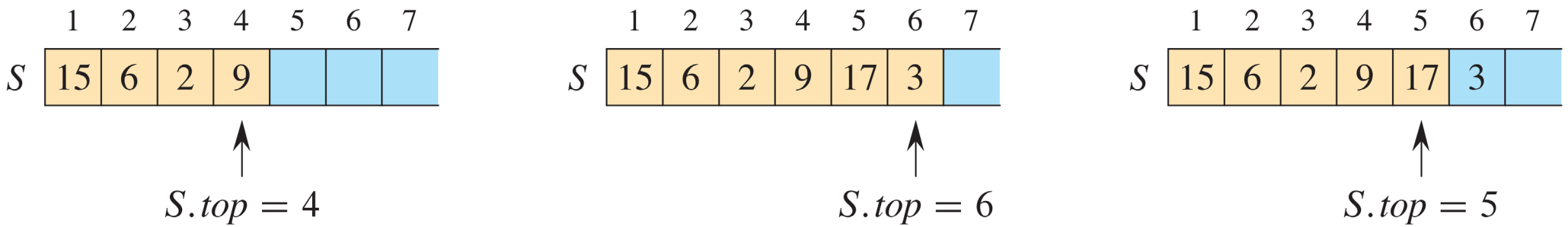



```
typedef struct {
    int col;     // Column index
    int value;   // Nonzero value
} Pair;
```

```
Pair **rows = (Pair **)malloc(numRows * sizeof(Pair *));
```

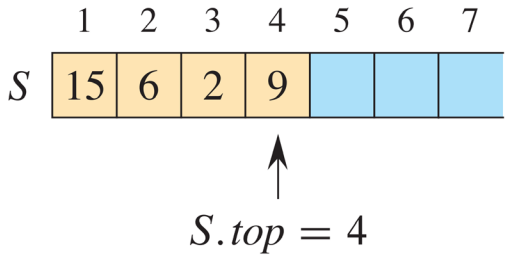
Stacks

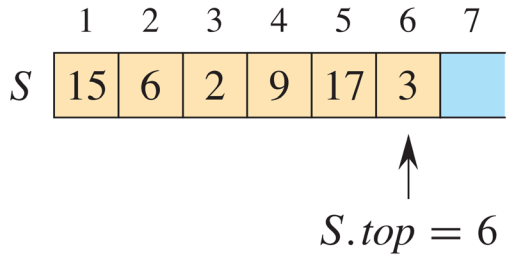
- Sometimes, the order in which we retrieve data is independent of its content, being only a function of when it arrived

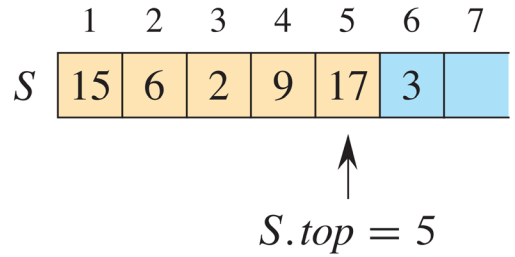


- A stack supports last-in, first-out operations:
  - Push(s,x) – Insert item x at the top of stack s.
  - Pop(s) – Return (and remove) the top item of stack s.

Stacks








```
STACK-EMPTY(S)
1  if S.top == 0
2        return TRUE
3  else return FALSE
```

```
PUSH(S, x)
1  if S.top == S.size
2        error "overflow"
3  else S.top = S.top + 1
4        S[S.top] = x
```

```
POP(S)
1  if STACK-EMPTY(S)
2        error "underflow"
3  else S.top = S.top − 1
4        return S[S.top + 1]
```

No searching or shifting of elements is required

Each of these operations takes O(1) time

Queues

- A queue supports first-in, first-out operations:
  - Enqueue(q,x) – Insert item x at the back of queue q.
  - Dequeue(q) – Return (and remove) the front item from queue q.

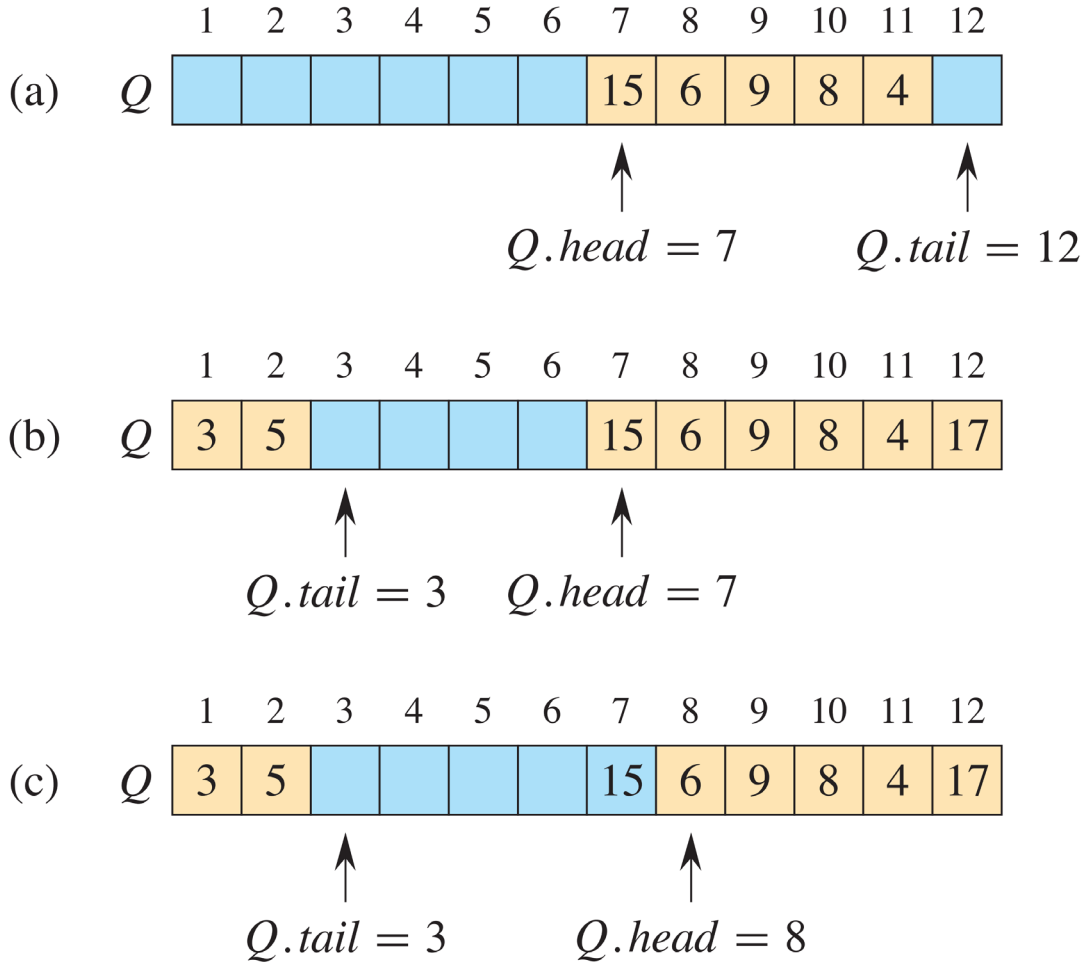
- Stacks are more easily represented as an array, with push/pop incrementing/decrementing a counter.

- Queues are more easily represented as a linked list, with enqueue/dequeue operating on opposite ends of the list.

Queues

```
ENQUEUE(Q, x)
1  Q[Q.tail] = x
2  if Q.tail == Q.size
3        Q.tail = 1
4  else Q.tail = Q.tail + 1

DEQUEUE(Q)
1  x = Q[Q.head]
2  if Q.head == Q.size
3        Q.head = 1
4  else Q.head = Q.head + 1
5  return x
```



Stacks and Queues

- Both are appropriate for a container class where order sometimes matter.

- Lines in banks are based on queues, while food in my refrigerator is treated as a stack.

- The entire difference between depth-first search (DFS) and breadth-first search (BFS) is whether a stack or a queue holds the vertices/items to be processed

Pointers and Linked Structures

- In C, a pointer p is assumed to give the address in memory where a particular chunk of data is located

- We use *p to denote the item that is pointed to by pointer p, and &x to denote the address of (i.e.pointer to) a particular variable x.

- A special NULL pointer value is used to denote structure-terminating or unassigned pointers.

List

```
typedef struct list {
        item_type item;
        struct list *next;
} list;
```

- Each node in our data structure (here list) contains one or more data fields (here item) that retain the data that we need to store.
- Each node contains a pointer field to at least one other node (here next). This means that much of the space used in linked data structures is devoted to pointers, not data.
- Finally, we need a pointer to the head of the structure, so we know where to access it.

Search List

```
list *search_list(list *l, item_type x) {
        if (l == NULL) {
                return(NULL);
        }

        if (l->item == x) {
                return(l);
        } else {
                return(search_list(l->next, x));
        }
}
```

Insert in a List

```
void insert_list(list **l, item_type x) {
        list *p;
        p = malloc(sizeof(list));
        p->item = x;
        p->next = *l;
        *l = p;
}
```

Delete in a List

```
void delete_list(list **l,  list  **x)  {
        list *p; /* item pointer */
        list *pred;       /* predecessor pointer */
        p = *l;
        pred = item_ahead(*l, *x);
        if  (pred  ==  NULL)  {
                *l  = p->next;
        }  else {
                pred->next = (*x)->next;
        }
        free(*x);         /* free memory used by node */
}
```

Advantages

- The advantages of linked structures over static arrays include:

- Overflow on linked structures never occurs unless the memory is actually full
- Insertion and deletion are simpler than for static arrays

- With large records, moving pointers is easier and faster than moving the items themselves

Disadvantages

- Conversely, the relative advantages of arrays include:
- Space efficiency: linked structures require extra memory for storing pointer fields.

- Efficient random access to items in arrays.

- Better memory locality and cache performance than random pointer jumping

Tree

- Natural representation for hierarchical information
  - the directory structure of a file system on a computer, parse trees in compilers, the structure of a knock-out sports tournament

- tree is used to design fast algorithms in computer science because of its efficiency relative to the simpler data structures
  - Operations that take linear time on these structures often take logarithmic time on an appropriately organized tree structure e.g. the average time complexity for a search on a key is linear on a linked list and logarithmic on a binary search tree

Types of Trees

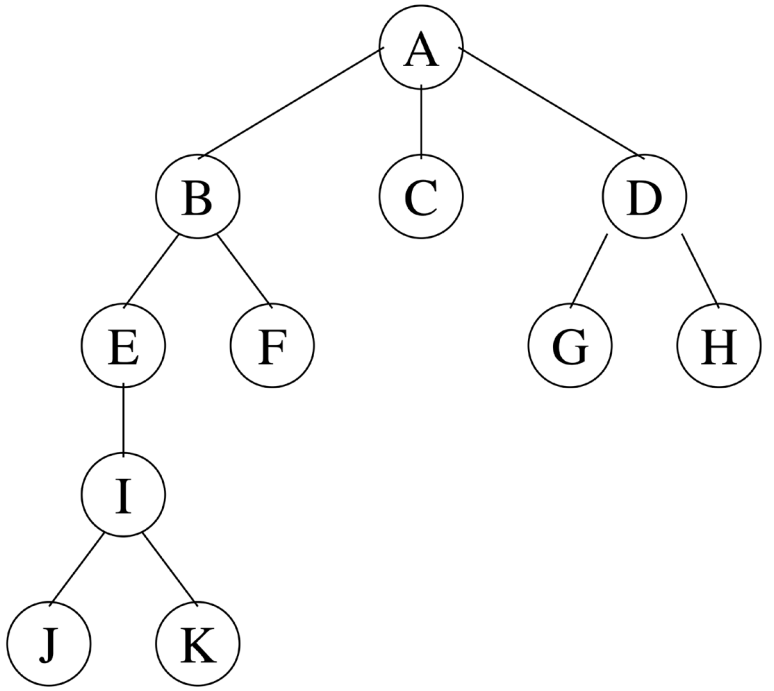
- Unrooted Tree: Tree without a root node or a sense of time
  - e.g. Minimum spanning tree of a graph
- Rooted Tree: There is a root node (start)
- Ordered Tree: children of each node are ordered
  - We often use Tree to mean Ordered Rooted Tree
- Binary Tree: Ordered tree with internal nodes of degree 2

Trees

- Degree: Number of subtrees of a node. Node with degree 0 are called leaf nodes
- Children: Roots of the subtrees of a node X are its children. X is the parent
- Siblings: Children of the same parent
- Level: If a node is at level l, its children are at level l+1
- Height: Maximum level of any node in the tree

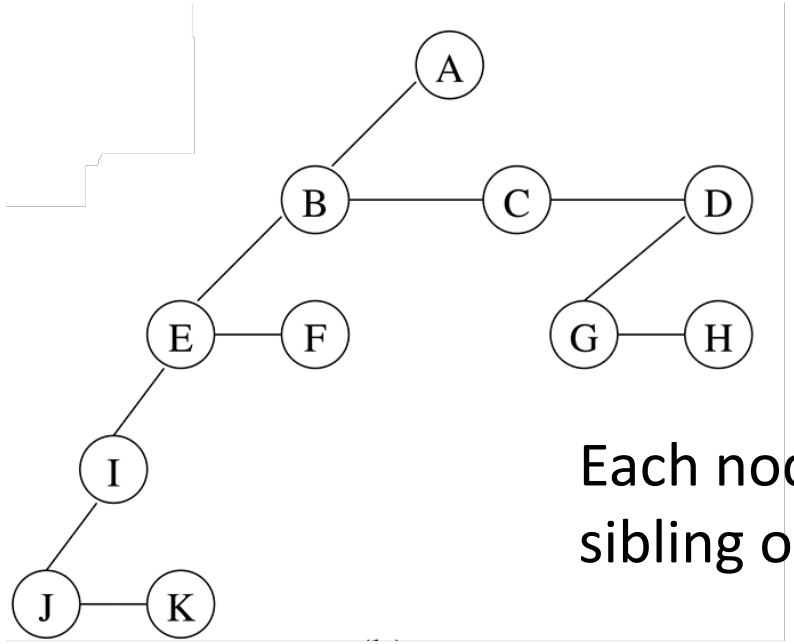
Tree Representation

- List: (A (B (E (I (J, K)), F), C, D(G, H)))



Each node has n array of pointers

- Left child-Right Sibling



Each node has a pointer to its leftmost child (if any) and to the sibling on its immediate right (if any)

Tree Representation

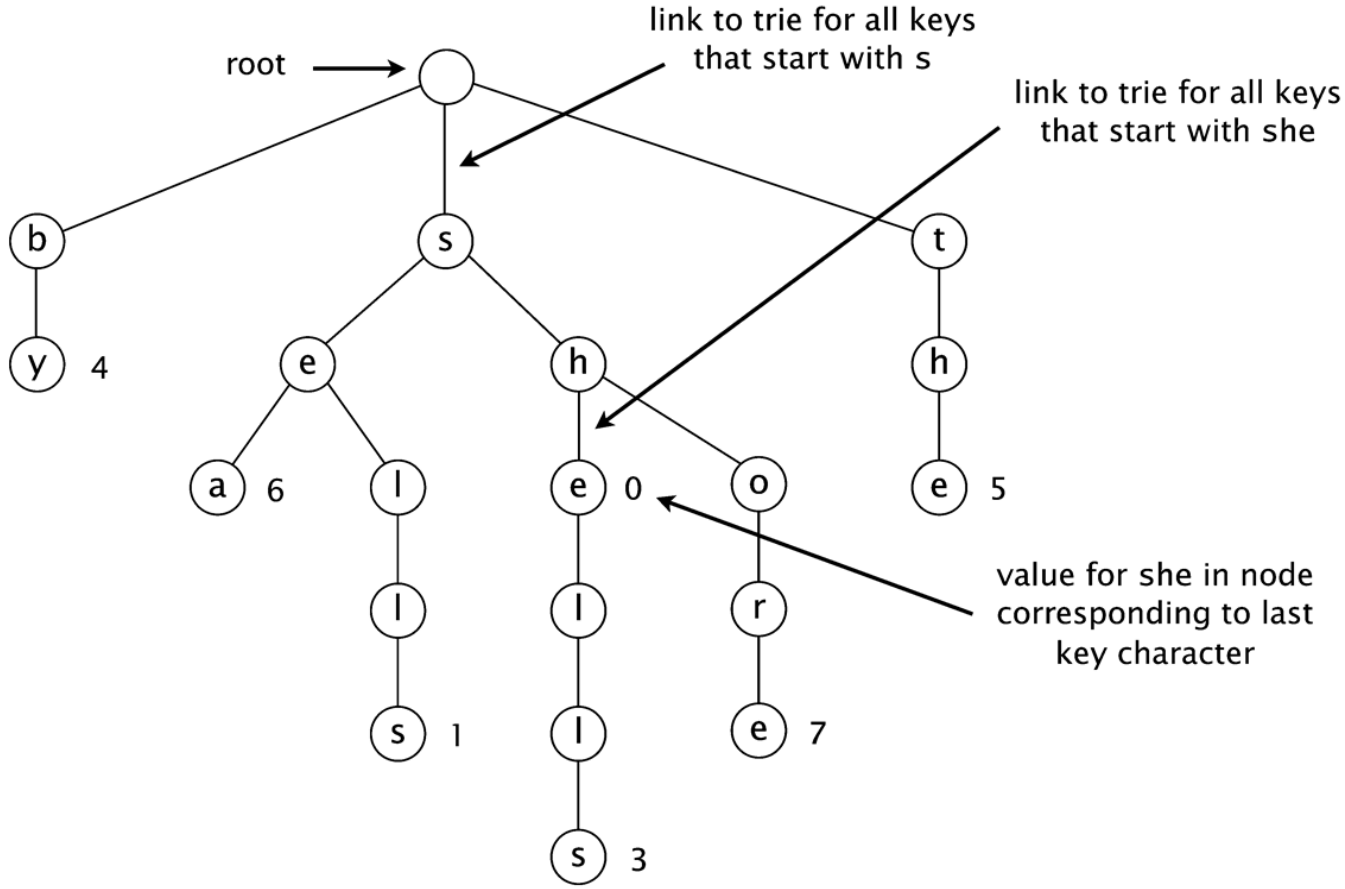
```
struct GNode {
  gpointer data;
  GNode  *next;
  GNode  *prev;
  GNode  *parent;
  GNode  *children;
};
```

```
typedef struct Node {
 gpointer data;
 struct Node *firstChild; // Pointer to the
first child
 struct Node *nextSibling; // Pointer to
the next sibling
} Node;
```

Tries

- Store characters in nodes
- Each node has R children, one for each possible character

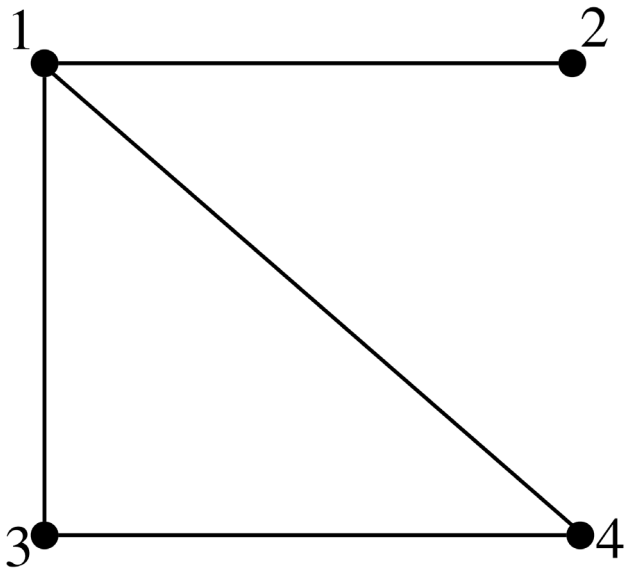
key	value
by	4
sea	6
sells	1
she	0
shells	3
shore	7
the	5

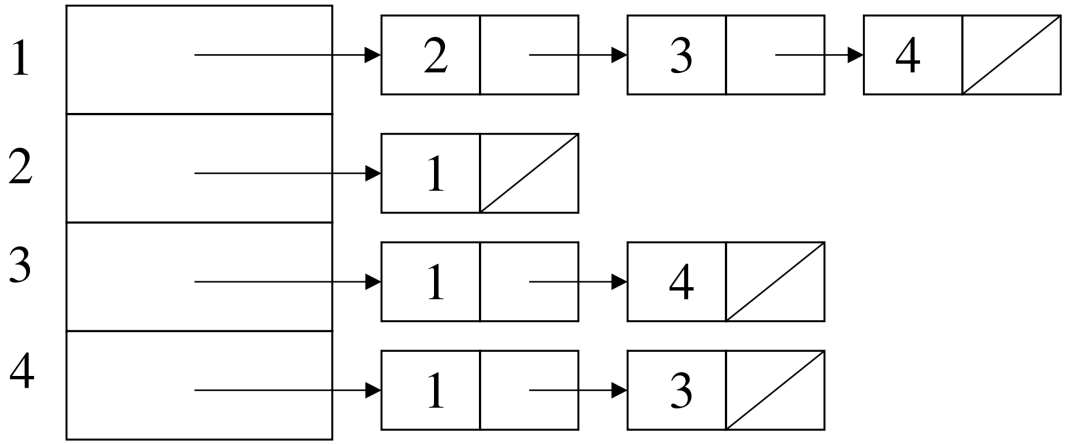


Graphs

- Tree can represent relationships that are hierarchical in nature
- Graph: Tree with edges between any pair of elements
- A graph G = (V, E) consists of a finite set of vertices $V = \{\upsilon_1, \upsilon_2, \ldots, \upsilon_n\}$ and a finite set E of edges $E = \{e_1, e_2, \ldots, e_m\}$
- undirected and directed (digraph): vertex pair (u,v) associated with an edge is ordered or unordered

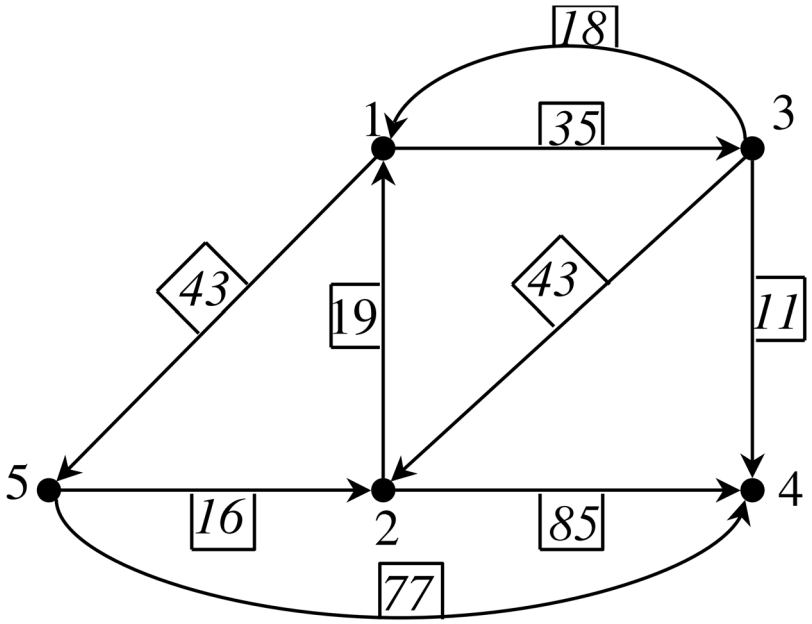
Graph Representation

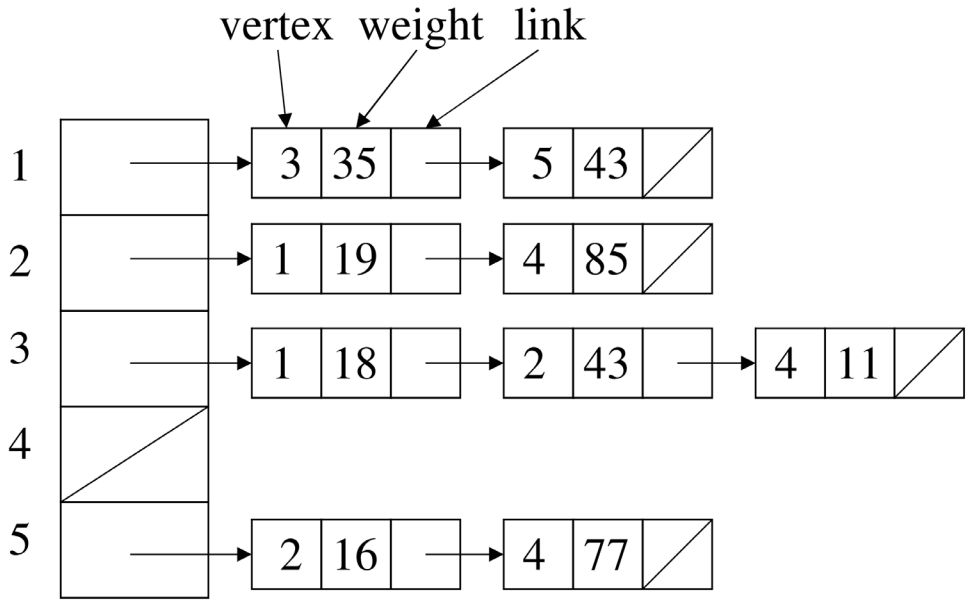




$$\begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

Weighted Graph





$$W = \begin{pmatrix} - & \infty & 35 & \infty & 43 \\ 19 & - & \infty & 85 & \infty \\ 18 & 43 & - & 11 & \infty \\ \infty & \infty & \infty & - & \infty \\ \infty & 16 & \infty & 77 & - \end{pmatrix}$$

Brainstorming

- Suppose we create a graph for X (Twitter), where a node is an account. The follower and following relationship form a directed edge. How could we determine influential accounts?

- Suppose you are a data scientist in a political science department. You decided to represent leaders as nodes in a graph. You decide to draw an edge if leader A has publicly endorsed leader B such as in interviews or writings. How could you identify tightly connected groups of leaders?

Ram Model of Computation

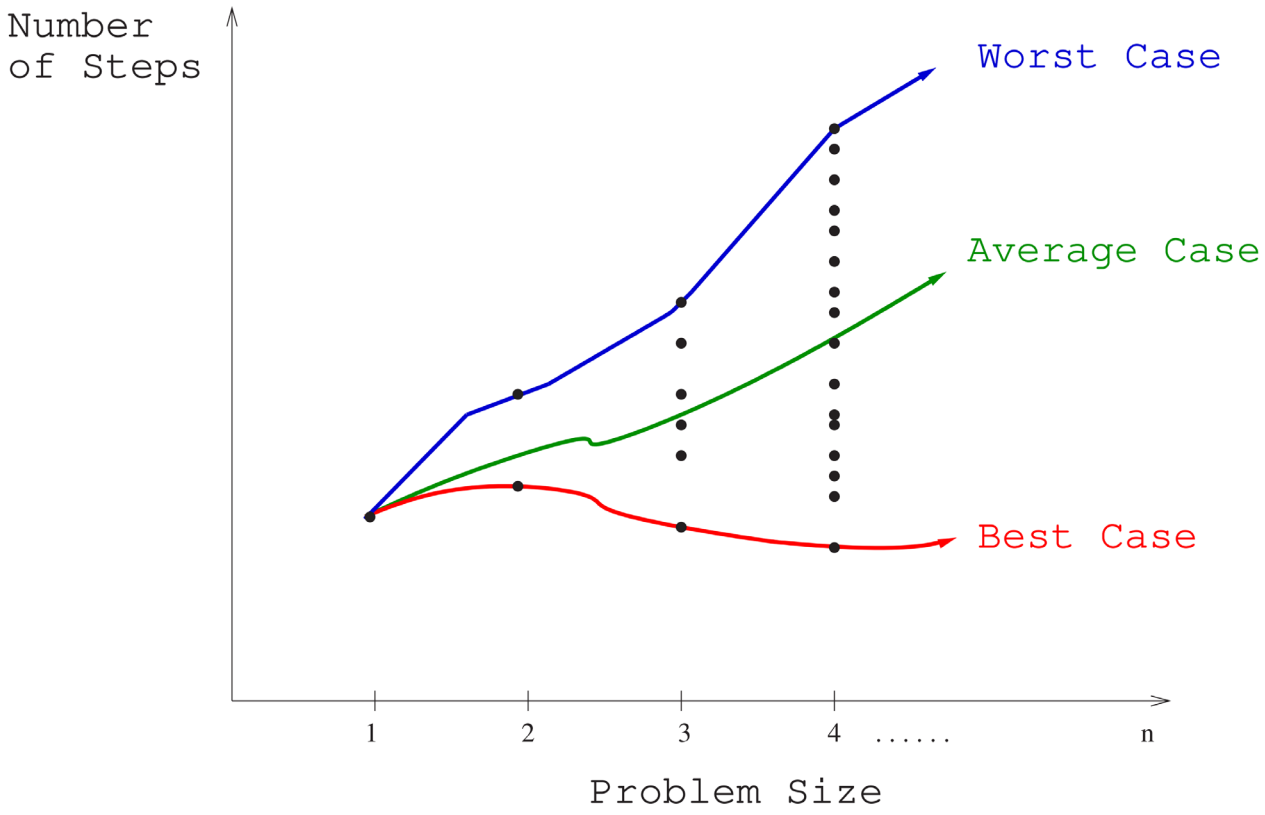
- Machine-independent algorithm design depends upon a hypothetical computer called the Random Access Machine, or RAM. Under this model of computation, we are confronted with a computer where:

- Each simple operation (+, *, –, =, if, call) takes exactly one time step

- Loops and subroutines are not considered simple operations. Instead, they are the composition of many single-step operations
  - sorting 1,000,000 items will certainly take much longer than sorting ten items

- Each memory access takes exactly one time step. Furthermore, we have as much memory as we need. The RAM model takes no notice of whether an item is in cache or on the disk

Best-Case, Average-Case, Worst-Case Complexity



Think about running an algorithm over all possible instances of data that can be fed to it. For the problem of sorting, the set of possible input instances includes every possible arrangement of *n* keys, for all possible values of *n*. We can represent each input instance as a point on a graph where the *x*-axis represents the size of the input problem (for sorting, the number of items to sort), and the *y*-axis denotes the number of steps taken by the algorithm.

Complexity

- We can define three interesting functions over the plot of these points:
- The **worst-case** complexity of the algorithm is the function defined by the maximum number of steps taken in any instance of size n. This represents the curve passing through the highest point in each column.

- The **best-case** complexity of the algorithm is the function defined by the minimum number of steps taken in any instance of size n. This represents the curve passing through the lowest point of each column.

- The **average-case** complexity or expected time of the algorithm, which is the function defined by the average number of steps over all instances of size n.

Complexity

Performing a precise worst-case analysis like : $T(n) = 12754n^2 + 4353n + 834 \lg_2 n + 13546$

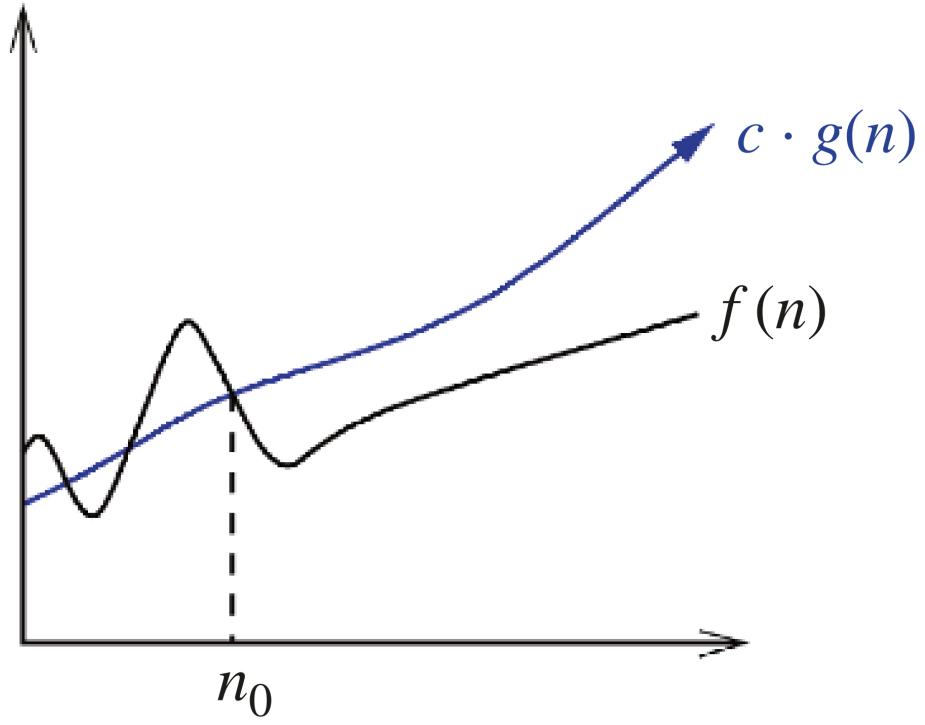
would clearly be very difficult work, but provides us little extra information than the observation that “the time grows quadratically with n.”

- It proves to be much easier to talk in terms of simple upper and lower bounds of time-complexity functions using the Big Oh notation. The Big Oh simplifies our analysis by ignoring levels of detail that do not impact our comparison of algorithms

Big Oh Notation

- The Big Oh notation ignores the difference between multiplicative constants. The functions f(n) = 2n and g(n) = n are identical in Big Oh analysis.
  - Why?

O-Notation



f(n) = O(g(n)) means c · g(n) is an upper bound on f(n). Thus, there exists some constant c such that f(n) ≤ c · g(n) for every large enough n (that is, for all n ≥ n_0, for some constant n_0)

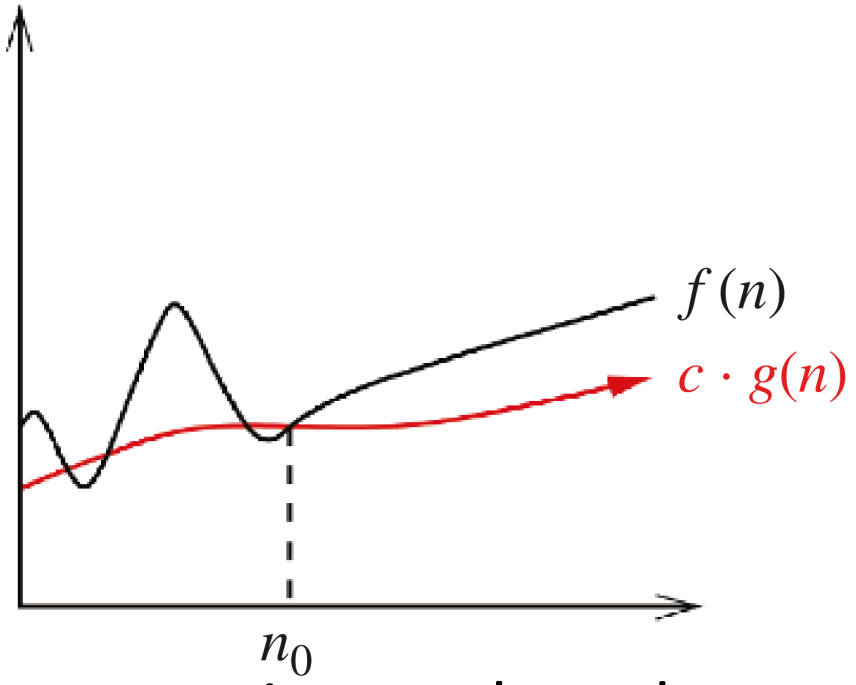
Example

$f(n) = 3n^2 - 100n + 6 = O(n^2)$, because for $c = 3$, $3n^2 > f(n)$;

$f(n) = 3n^2 - 100n + 6 = O(n^3)$, because for $c = 1$, $n^3 > f(n)$ when $n > 3$;

$f(n) = 3n^2 - 100n + 6 \neq O(n)$, because for any $c > 0$, $c.n < f(n)$ when $n > (c + 100)/3$, since $n > (c + 100)/3 \Rightarrow 3n > c + 100 \Rightarrow 3n^2 > cn + 100n > cn + 100n - 6 \Rightarrow 3n^2 - 100n + 6 = f(n) > cn$;

Ω Notation



- Ω -notation characterizes a lower bound

on the asymptotic behavior of a function

- It says that a function grows at least as fast as a certain rate, based - as in O-notation - on the highest-order term.

- The function $7n^3 + 100n^2 – 20$ grows at least as fast as n^3: $\Omega(n^3)$
- The function grows at least as fast as n^2 also: $\Omega(n^2)$
  - It is $\Omega(n^c)$ for $c \leq 3$

Example

$f(n) = 3n^2 - 100n + 6 = \Omega(n^2)$, because for $c = 2$, $2n^2 < f(n)$ when $n>100$

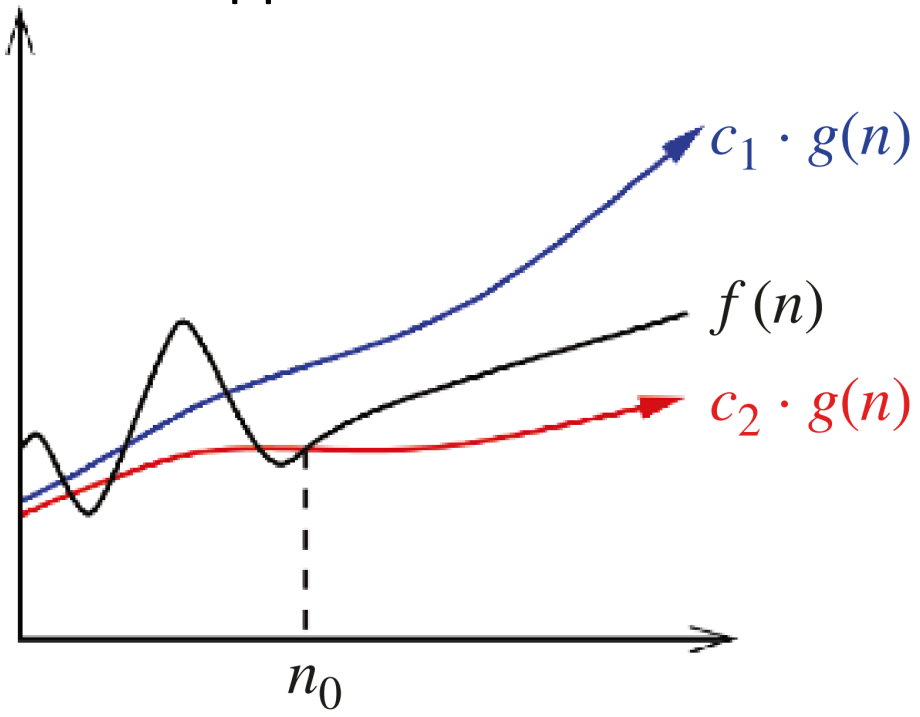
$f(n) = 3n^2 - 100n + 6 \neq \Omega(n^3)$, because for $c > 0$, $f(n) < c.n^3$ when $n > (3/c+3)$;

$f(n) = 3n^2 - 100n + 6 = \Omega(n)$, because for any $c > 0$, $f(n) < cn^3$ when $n>\max(9/c,1)$;

Θ-notation

- Θ-notation characterizes a tight bound on the asymptotic behaviour of a function
  - Still based on the highest ordered term but with a upper and lower bound

- If you can show that a function is both O(f(n)) and Ω(f(n)) for some function f(n), then you have shown that the function is , Θ(f(n))
  - For example, $7n^3 + 100n^2 – 20$



Example

$f(n) = 3n^2 - 100n + 6 = \Theta(n^2)$, because both O and Ω apply

$f(n) = 3n^2 - 100n + 6 \neq \Theta(n^3)$, because only O applies

$f(n) = 3n^2 - 100n + 6 \neq \Theta(n)$, because only Ω applies

Question

- Is $2^{n+1} = O(2^n)$?
- Is $2^{n+1} = \Omega(2^n)$?
- Is $2^{n+1} = \Theta(2^n)$?

Running Times

- Assume one operation costs 1 nanosecond, 10^{-9} seconds

n	$\lg n$	n	$n \lg n$	n^2	2^n	$n!$
10	0.003 μs	0.01 μs	0.033 μs	0.1 μs	1 μs	3.63 ms
20	0.004 μs	0.02 μs	0.086 μs	0.4 μs	1 ms	77.1 years
30	0.005 μs	0.03 μs	0.147 μs	0.9 μs	1 sec	8.4×10^{15} yrs
40	0.005 μs	0.04 μs	0.213 μs	1.6 μs	18.3 min	
50	0.006 μs	0.05 μs	0.282 μs	2.5 μs	13 days	
100	0.007 μs	0.1 μs	0.644 μs	10 μs	4×10^{13} yrs	
1,000	0.010 μs	1.00 μs	9.966 μs	1 ms		
10,000	0.013 μs	10 μs	130 μs	100 ms		
100,000	0.017 μs	0.10 ms	1.67 ms	10 sec		
1,000,000	0.020 μs	1 ms	19.93 ms	16.7 min		
10,000,000	0.023 μs	0.01 sec	0.23 sec	1.16 days		
100,000,000	0.027 μs	0.10 sec	2.66 sec	115.7 days		
1,000,000,000	0.030 μs	1 sec	29.90 sec	31.7 years		

Question

- What is the big-oh complexity of binary search?

- What is the time complexity of:
  - select * from users
  - select * from users where email = “xyz@abc.com”

- What is the complexity if an index exists on the email column?

Divide and Conquer

- Break problems into smaller, easy-to-solve pieces
- Each subproblem is solved independently
- Finally, the solutions to the subproblems are combined in order to obtain the solution to the original problem
- Often implemented using recursion

Binary search

```
int binary_search(item_type s[],
item_type key, int low, int high) {
int middle; /* index of middle element
*/
if (low > high) {
  return (-1); /* key not found */
}

middle = (low + high) / 2;
if (s[middle] == key) {
  return(middle);
}
if (s[middle] > key) {
  return(binary_search(s, key, low, middle -
1));
} else {
  return(binary_search(s, key, middle + 1,
high));
}
}
```

Recurrence Relation

- It is an equation where a function is defined in terms of itself
- Factorial: $a_1 = 1$, $a_n = n \cdot a_{n-1}$
- Fibonacci: $a_0=1$, $a_1=1$, $a_n=a_{n-1}+a_{n-2}$
- Exponential: $a_n = 2a_{n-1}$, $a_1 = 1$

Divide and Conquer Recurrences

- A typical divide-and-conquer algorithm breaks a given problem into ‘a’ smaller pieces, each of which is of size n/b. It then spends f(n) time to combine these subproblem solutions into a complete result

- Let T (n) denote the worst-case time this algorithm takes to solve a problem of size n. Then T (n) is given by the following recurrence relation:

$$T(n) = a \cdot T(n/b) + f(n)$$

Binary Search

- The running time of binary search is governed by the recurrence

$T(n) = T(n/2) + O(1)$

- since at each step we spend constant time to reduce the problem to an instance half its size

- This recurrence evaluates to $T(n) = O(\lg n)$

Mergesort

- The running time of mergesort is governed by the recurrence

$T(n) = 2T(n/2) + O(n)$

- since the algorithm divides the data into equal sized halves and then spends linear time merging the halves after they are sorted

- this recurrence evaluates to $T(n) = O(n \lg n)$

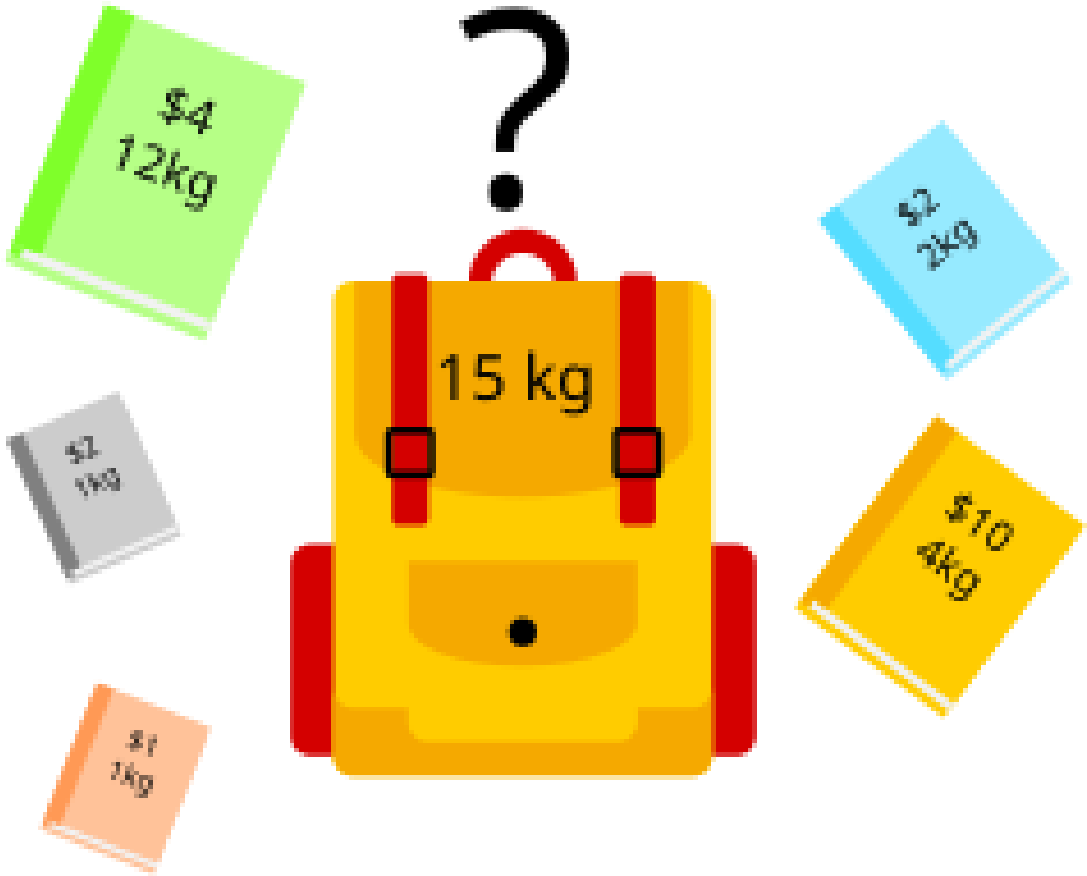
Greedy Algorithm

- Algorithms for optimization problems typically go through a sequence of steps, with a set of choices at each step

- A greedy algorithm always makes the choice that looks best at the moment
  - makes a locally optimal choice in the hope that this choice leads to a globally optimal solution

Knapsack Problem

Given a set of items, each with a weight and a value, determine which items to include in the collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.



Questions

- How many possible solutions are there?

- What is the worst-case time complexity of a brute-force solution?

Greedy Strategies

- **Greedy by Profit:** At each step select from the remaining items the one with the highest profit. Chooses the most profitable items first

- **Greedy by Weight:** At each step select from the remaining items the one with the least weight. Tries to maximize the profit by putting as many items into the knapsack as possible

- **Greedy by Profit Density:** At each step select from the remaining items the one with the largest profit density, pi/ wi. Tries to maximize the profit by choosing items with the largest profit per unit of weight

Knapsack - Greedy

- Assume W=100

				greedy by			
i	w_i	p_i	p_i/w_i	profit	weight	density	optimal solution
1	100	40	0.4	yes	no	no	no
2	50	35	0.7	no	no	yes	yes
3	45	18	0.4	no	yes	no	yes
4	20	4	0.2	no	yes	yes	no
5	10	10	1.0	no	yes	yes	no
6	5	2	0.4	no	yes	yes	yes
total weight				100	80	85	100
total profit				40	34	51	55

Question

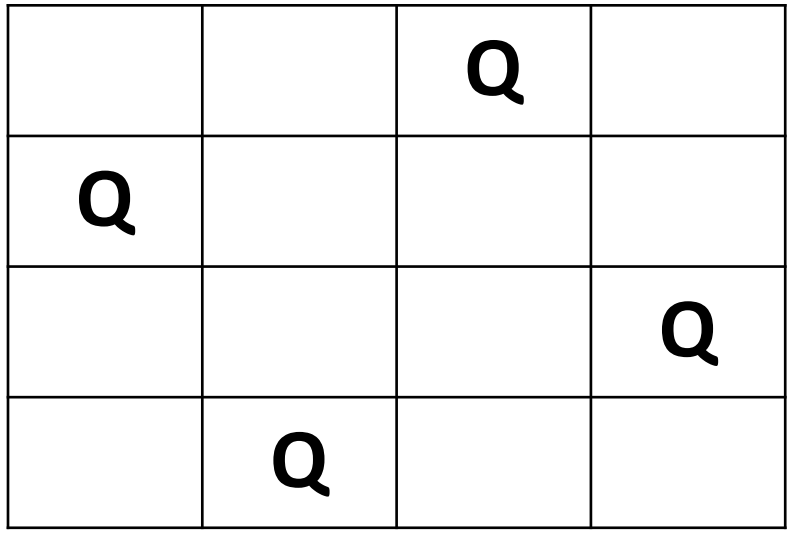
- What are the time complexities?

Backtracking

- Systematic way to run through all the possible configurations of a search space
  - all arrangements of objects (permutation)
  - all subsets
  - all paths between two vertices etc.

- At each step in the backtracking algorithm, we try to extend a given partial solution $a = (a_1, a_2, ..., a_k)$ by adding another element at the end. After this extension, we must test whether what we now have is a complete solution

n-Queens Problem



The problem is to place n queens (chess pieces) on an n by n board so that no two queens are in the same row, column or diagonal

n-Queens: Backtracking

```
rnQueens (k,n) {
  for row [ k]= 1 to n
  if (positionOK( k) )
    if ( k = n ) {
          for i = 1 to n
            print (row [ i] + " "); println;
    } else
          rnQueens ( k+1, n )
}
```

```
positionOK( k) {
  for i = 1 to k – 1
    if (row [ k] = row [ i] ||
    abs (row [ k] –row [ i]) = k –i )
          return false; return true;
}
```

Branch and Bound

- Branching:
  - Divide the problem into smaller subproblems
  - Create a search tree representing potential solutions
  - Each node represents a partial solution

- Bounding:
  - Calculate a bound for each branch.
  - Prune branches that cannot yield better solutions than the current best.

- Pruning:
  - Eliminate branches to save time and resources.

Branch and Bound

- Often used to solve combinatorial optimization problems
  - minimize f(x) subject to x ∈ X

- Consider the Travelling Salesman Problem. We can frame it as an optimization problem:

Let $y_{ij} \in \{0, 1\}$ for i, j = 1, . . . , n [y_{ij}=1 if the tour traverses the edge (i,j) 0 otherwise]

The objective function is then given by:

$\min \sum_i \sum_j c_{ij} y_{ij}$ [c_{ij} is the distance between cities]

Constraints

- Should visit only one city after i

$\sum_j y_{ij} = 1$ for i= 1,2,3,…,n-1

- Should have come from only one city to j

$\sum_i y_{ij} = 1$ for j= 1,2,3,…,n-1

Travelling Salesman Problem

1. **Initialization**:
   - Start with a partial route (e.g., just the starting city).
   - Calculate the lower bound for completing this route.

2. **Branching**:

   Extend the route by visiting each unvisited city, creating new subproblems.

TSP

3. Bounding:

- For each extended route, calculate a lower bound based on the remaining unvisited cities. Example: Current route: A -> B. Remaining cities: C, D.

- Lower bound:
  Cost(A -> B) + Minimum spanning tree of {C, D}
  + Cost(B -> nearest city in {C, D})

- Upper bound:
  Cost of the best tour so far

TSP

4. **Pruning:**
   - Discard branches with bounds higher than the current best solution. Example: If the bound for a route exceeds the shortest known route, stop exploring that branch.

5. **Selection:**
   - Continue exploring the branch with the lowest bound.

Randomized Algorithm

- **Deterministic Algorithm**: The running time and the output are functions of the input only
  - Always follows the same execution path for the same input

- Randomized algorithm also includes random values

- Example: Finding a minimum cut

Minimum Cut

- Input: An undirected connected graph G = (V, E)
- A cut in a graph is a partition of V into two sets S and (V – S) denoted as (S, V–S)
- Size of a cut is the number of edges with one endpoint in S and the other endpoint in V – S
- Minimum cut: The cut with the minimum size

Karger's Algorithm

- A simple and elegant Monte Carlo randomized algorithm that works faster

While |V| > 2

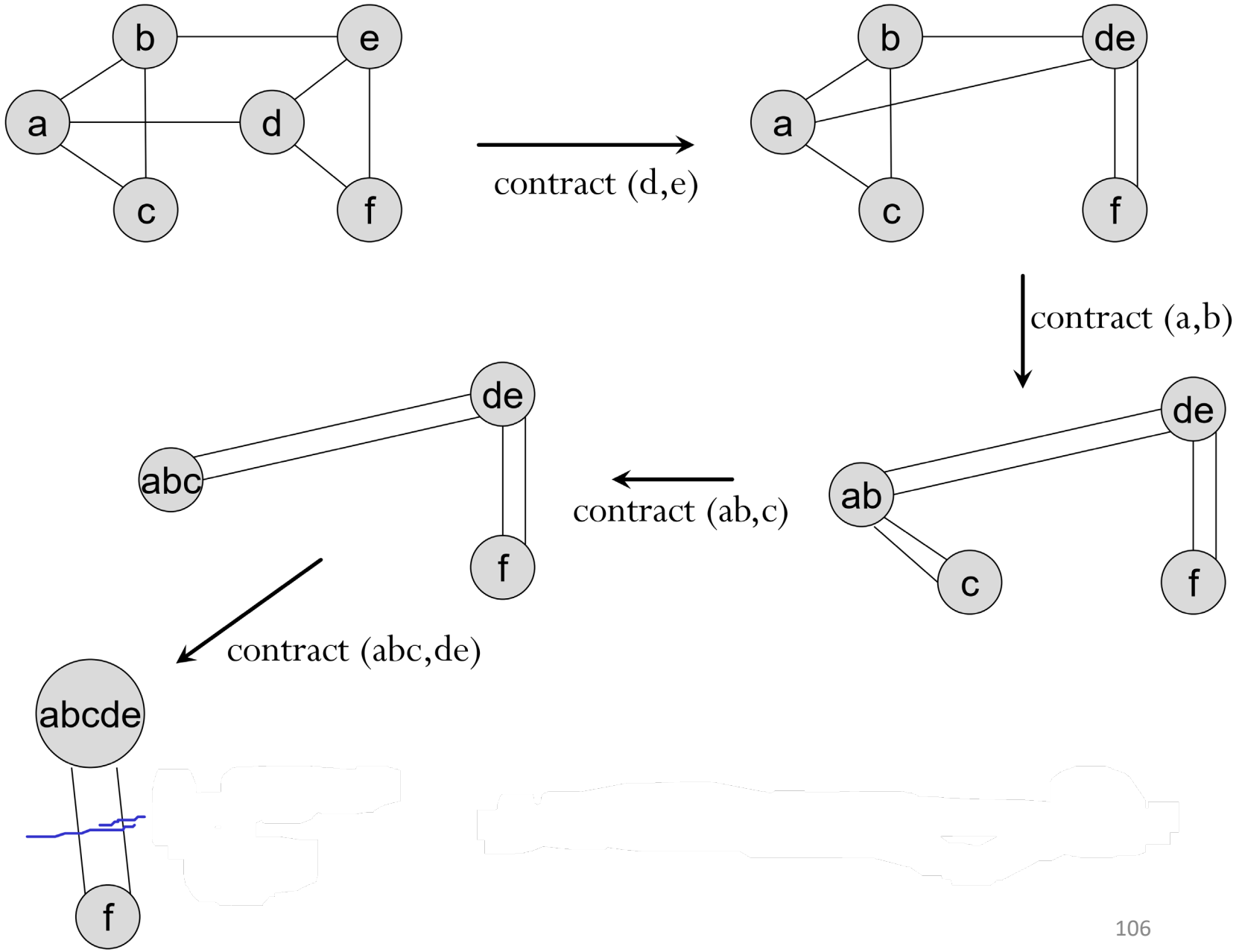
Randomly pick an edge (u,v) in E

Contract u and v into a single vertex uv

Return the set of edges in E between the two final vertices left as the minimum cut

- What does contracting (u, v) mean? : Add a node uv to V and remove nodes u and v from V. So, V = V – {u, v} + uv

Example



Min Cut =

({abcde}, {f})